**Lab 9 Report — Concurrent TCP Server in Go**

**LINK TO GITHUB :**

1. **Cause of Crash**

The main reason that the server crash is because of concurrency and resource exhaustion, not really from the network problem. When many clients connect at the same time, the server creates a lot of goroutines and keeps many connections open. This makes CPU and memory usage increase, and also makes the system harder to manage.

Also, some clients may be slow or inactive, but still keep the connection open. If there is no timeout or limit, these connections will waste resources. In the end, the server can become slow or even stop responding.

---

2. **Debugging Evidence**

```
20:22:51.636474 [RECV] 127.0.0.1:55882 → "LOAD_TEST_PING"
20:22:51.636478 [CONNECT] Client: 127.0.0.1:55890 | Active: 78
20:22:51.636479 [CONNECT] Client: 127.0.0.1:55892 | Active: 79
20:22:51.636487 [CONNECT] Client: 127.0.0.1:55893 | Active: 81
20:22:51.636486 [CONNECT] Client: 127.0.0.1:55888 | Active: 80
20:22:51.636494 [CONNECT] Client: 127.0.0.1:55894 | Active: 82
20:22:51.636474 [CONNECT] Client: 127.0.0.1:55891 | Active: 76
20:22:51.636414 [CONNECT] Client: 127.0.0.1:55886 | Active: 75
20:22:51.636496 [RECV] 127.0.0.1:55893 → "LOAD_TEST_PING"
20:22:51.636476 [CONNECT] Client: 127.0.0.1:55889 | Active: 77
20:22:51.636523 [RECV] 127.0.0.1:55880 → "LOAD_TEST_PING"
20:22:51.636522 [CONNECT] Client: 127.0.0.1:55896 | Active: 84
20:22:51.636531 [RECV] 127.0.0.1:55881 → "LOAD_TEST_PING"
20:22:51.636512 [CONNECT] Client: 127.0.0.1:55895 | Active: 83
20:22:51.636526 [CONNECT] Client: 127.0.0.1:55897 | Active: 85
20:22:51.636549 [CONNECT] Client: 127.0.0.1:55899 | Active: 87
20:22:51.636556 [RECV] 127.0.0.1:55899 → "LOAD_TEST_PING"
20:22:51.636561 [RECV] 127.0.0.1:55884 → "LOAD_TEST_PING"
20:22:51.636565 [CONNECT] Client: 127.0.0.1:55902 | Active: 88
20:22:51.636581 [RECV] 127.0.0.1:55902 → "LOAD_TEST_PING"
20:22:51.636582 [CONNECT] Client: 127.0.0.1:55903 | Active: 89
20:22:51.636605 [CONNECT] Client: 127.0.0.1:55904 | Active: 90
20:22:51.636536 [CONNECT] Client: 127.0.0.1:55898 | Active: 86
20:22:51.636615 [CONNECT] Client: 127.0.0.1:55905 | Active: 91
20:22:51.636619 [RECV] 127.0.0.1:55898 → "LOAD_TEST_PING"
20:22:51.636628 [CONNECT] Client: 127.0.0.1:55908 | Active: 92
20:22:51.636633 [CONNECT] Client: 127.0.0.1:55909 | Active: 93
20:22:51.636629 [RECV] 127.0.0.1:55897 → "LOAD_TEST_PING"
20:22:51.636625 [RECV] 127.0.0.1:55895 → "LOAD_TEST_PING"
20:22:51.636658 [CONNECT] Client: 127.0.0.1:55912 | Active: 97
20:22:51.636622 [RECV] 127.0.0.1:55896 → "LOAD_TEST_PING"
20:22:51.636688 [RECV] 127.0.0.1:55889 → "LOAD_TEST_PING"
20:22:51.636641 [CONNECT] Client: 127.0.0.1:55901 | Active: 94
20:22:51.636651 [RECV] 127.0.0.1:55879 → "LOAD_TEST_PING"
20:22:51.636707 [RECV] 127.0.0.1:55894 → "LOAD_TEST_PING"
20:22:51.636655 [RECV] 127.0.0.1:55891 → "LOAD_TEST_PING"
20:22:51.636716 [CONNECT] Client: 127.0.0.1:55907 | Active: 99
20:22:51.636718 [CONNECT] Client: 127.0.0.1:55906 | Active: 100
20:22:51.636722 [RECV] 127.0.0.1:55888 → "LOAD_TEST_PING"
20:22:51.636732 [RECV] 127.0.0.1:55887 → "LOAD_TEST_PING"
20:22:51.636655 [CONNECT] Client: 127.0.0.1:55910 | Active: 95
20:22:51.636744 [RECV] 127.0.0.1:55909 → "LOAD_TEST_PING"
20:22:51.636657 [CONNECT] Client: 127.0.0.1:55911 | Active: 96
20:22:51.636704 [RECV] 127.0.0.1:55890 → "LOAD_TEST_PING"
20:22:51.636714 [CONNECT] Client: 127.0.0.1:55900 | Active: 98
20:22:51.636769 [RECV] 127.0.0.1:55886 → "LOAD_TEST_PING"
20:22:51.636781 [RECV] 127.0.0.1:55908 → "LOAD_TEST_PING"
20:22:51.636791 [RECV] 127.0.0.1:55904 → "LOAD_TEST_PING"
20:22:51.636796 [RECV] 127.0.0.1:55903 → "LOAD_TEST_PING"
20:22:51.636801 [RECV] 127.0.0.1:55905 → "LOAD_TEST_PING"
20:22:51.636837 [RECV] 127.0.0.1:55892 → "LOAD_TEST_PING"
20:22:51.636852 [RECV] 127.0.0.1:55910 → "LOAD_TEST_PING"
20:22:51.636858 [RECV] 127.0.0.1:55907 → "LOAD_TEST_PING"
20:22:51.636868 [RECV] 127.0.0.1:55911 → "LOAD_TEST_PING"
20:22:51.636870 [RECV] 127.0.0.1:55901 → "LOAD_TEST_PING"
20:22:51.636883 [RECV] 127.0.0.1:55906 → "LOAD_TEST_PING"
20:22:51.636895 [RECV] 127.0.0.1:55900 → "LOAD_TEST_PING"
```

**S**

From the server logs during load testing, we can clearly see that many clients are connecting at the same time. For example:

[CONNECT] Client: 127.0.0.1:55890 | Active: 78
[CONNECT] Client: 127.0.0.1:55892 | Active: 79
[CONNECT] Client: 127.0.0.1:55894 | Active: 82
[CONNECT] Client: 127.0.0.1:55902 | Active: 88
[CONNECT] Client: 127.0.0.1:55905 | Active: 91

[CONNECT] Client: 127.0.0.1:55909 | Active: 93
[CONNECT] Client: 127.0.0.1:55906 | Active: 100

At the same time, the server is also receiving data from multiple clients:

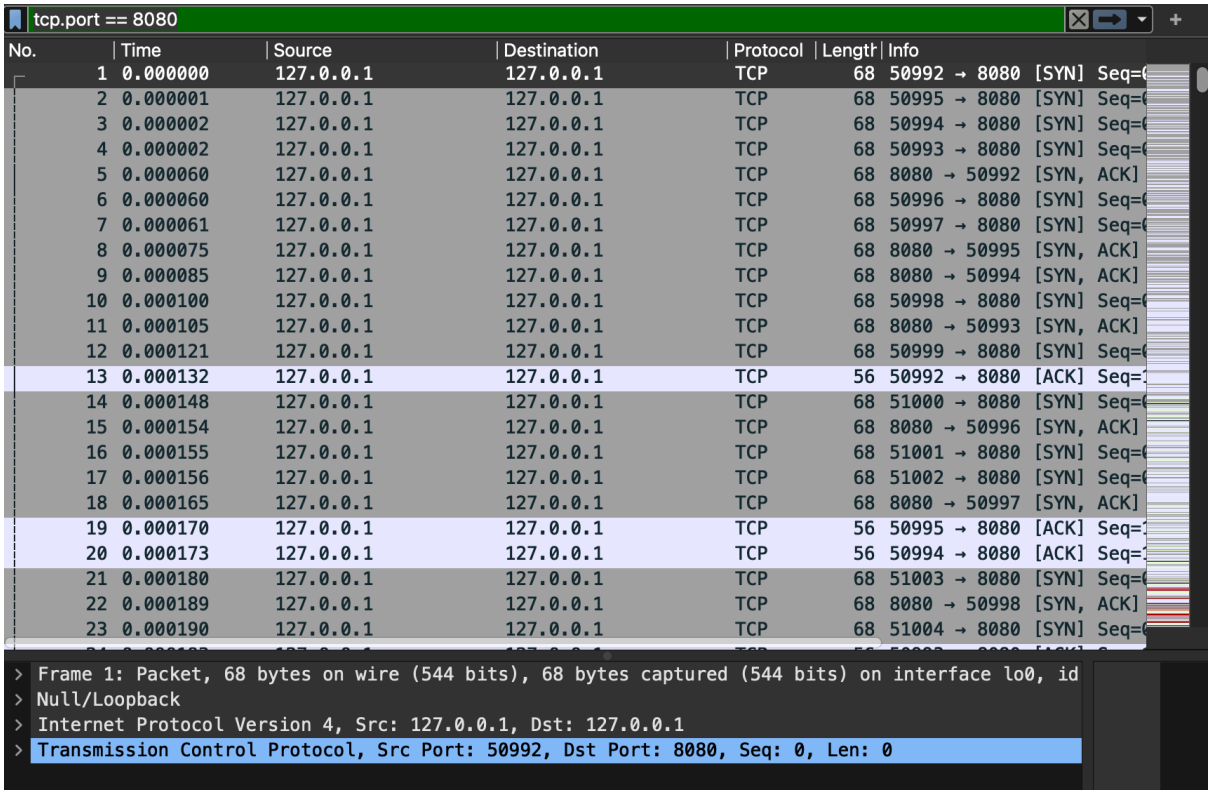
[RECV] 127.0.0.1:55880 -> "LOAD_TEST_PING"
[RECV] 127.0.0.1:55883 -> "LOAD_TEST_PING"
[RECV] 127.0.0.1:55902 -> "LOAD_TEST_PING"

This shows that:

- many clients are connected simultaneously
- active connections increase rapidly (up to around 100)
- the server is handling multiple requests at the same time

This confirms that the server is under high concurrency, which increases resource usage and can lead to performance issues or crashes.

WiresharkAnalysis



tcp.port == 8080

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50992 → 8080 [SYN] Seq=0 |
| 2 | 0.000001 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50995 → 8080 [SYN] Seq=0 |
| 3 | 0.000002 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50994 → 8080 [SYN] Seq=0 |
| 4 | 0.000002 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50993 → 8080 [SYN] Seq=0 |
| 5 | 0.000060 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50992 [SYN, ACK] |
| 6 | 0.000060 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50996 → 8080 [SYN] Seq=0 |
| 7 | 0.000061 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50997 → 8080 [SYN] Seq=0 |
| 8 | 0.000075 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50995 [SYN, ACK] |
| 9 | 0.000085 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50994 [SYN, ACK] |
| 10 | 0.000100 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50998 → 8080 [SYN] Seq=0 |
| 11 | 0.000105 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50993 [SYN, ACK] |
| 12 | 0.000121 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 50999 → 8080 [SYN] Seq=0 |
| 13 | 0.000132 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 50992 → 8080 [ACK] Seq=1 |
| 14 | 0.000148 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 51000 → 8080 [SYN] Seq=0 |
| 15 | 0.000154 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50996 [SYN, ACK] |
| 16 | 0.000155 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 51001 → 8080 [SYN] Seq=0 |
| 17 | 0.000156 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 51002 → 8080 [SYN] Seq=0 |
| 18 | 0.000165 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50997 [SYN, ACK] |
| 19 | 0.000170 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 50995 → 8080 [ACK] Seq=1 |
| 20 | 0.000173 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 50994 → 8080 [ACK] Seq=1 |
| 21 | 0.000180 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 51003 → 8080 [SYN] Seq=0 |
| 22 | 0.000189 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 8080 → 50998 [SYN, ACK] |
| 23 | 0.000190 | 127.0.0.1 | 127.0.0.1 | TCP | 68 | 51004 → 8080 [SYN] Seq=0 |

```
> Frame 1: Packet, 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface lo0, id
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 50992, Dst Port: 8080, Seq: 0, Len: 0
```

To analyze the network behavior, Wireshark was used with the filter:

tcp.port == 8080

From the captured packets, it can be seen that many TCP connections are happening almost at the same time. There are multiple SYN packets sent from different source ports to port 8080, followed by SYN-ACK and ACK packets, which shows that the TCP handshake is happening in parallel.

This means that multiple clients are trying to connect to the server at the same time, and each connection uses a different source port. The server is handling these connections concurrently.

Also, the large number of SYN packets clearly shows that many connection requests are happening at once. This increases the workload of the server and can lead to higher resource usage.

Overall, the Wireshark result supports that the issue is caused by high concurrency and a large number of connections, not just a network problem.

## 3. Metrics

The important metrics to monitor are:

1. Active connections
   This shows how many clients are connected at the same time
2. Goroutine count
   This shows how many concurrent tasks are running in the system
3. Error or timeout rate
   This shows how often the server has failed or timeout connections

Other useful metrics can be CPU usage, memory usage, and response time.

---

## 4. Mitigation Strategy

To reduce the problem, several solutions are used:

1. Connection limiting
   Use a channel (semaphore) to limit how many clients can connect at the same time
2. Timeout
   Set a deadline for each connection so slow clients will not stay forever
3. Input validation
   Check input size and ignore invalid or too large data
4. Logging
   Log connection activity to help debug and monitor the system

These methods help reduce resource usage and make the server more stable.

---

## 5. Conclusion

In conclusion, the crash is mainly caused by concurrency and resource management problem, not just network issue. From the logs and testing, we can see that many connections and goroutines increase under load. By using connection limit, timeout, and validation, the server can handle load better and avoid crashing.